CS 470 Final Reflection

Rakim Spriggs

August 2026

CS 470 Final Reflection

Presentation Video: https://youtu.be/gK9zqFPckas

Experiences and Strengths

CS 470 helped me develop a stronger understanding of how full-stack applications can be moved into cloud environments and how different cloud services can work together. Before this course, I had experience with programming, web development, databases, and other software development concepts, but this course gave me a better understanding of how those skills connect to cloud infrastructure.

One of the most valuable skills I developed was containerization. I learned how Docker can be used to package an application and its dependencies into containers. I also learned how Docker Compose can manage multiple services and allow them to communicate with each other. Working with the web application, API, and MongoDB as separate services helped me better understand how applications can be broken into smaller components.

I also developed experience with AWS services such as Amazon S3, API Gateway, and Lambda. These services helped me understand the difference between running an application on a traditional server and using cloud services that can scale based on demand. I also learned about concepts such as elasticity, pay-for-use pricing, IAM roles, policies, and cloud security.

These skills make me a more marketable candidate because modern software development increasingly involves cloud technologies. I can now discuss not only how to write application code, but also how that application can be packaged, deployed, secured, and scaled.

One of my strengths as a software developer is problem solving. During this course, I encountered issues involving development environments, Docker, Node versions, Angular builds, networking, and cloud configuration. Instead of stopping when something did not

work, I learned to troubleshoot the problem, identify the cause, and make changes until the application worked correctly.

Another strength is my ability to learn new technologies. CS 470 required me to work with technologies and concepts that were different from simply writing application code. I had to understand containers, orchestration, cloud storage, serverless computing, APIs, databases, and security. Being able to learn these technologies and connect them together is a skill I can continue using throughout my career.

Based on the skills developed in this course and my previous programming experience, I am prepared to pursue roles such as a software developer, full-stack developer, backend developer, cloud-focused developer, or junior cloud engineer. I am also interested in positions where I can continue developing my skills in AWS, automation, DevOps, and cloud-native application development.

Planning for Growth

One of the most important lessons from CS 470 is that an application should be designed with future growth in mind. A web application may work well with a small number of users, but the architecture needs to be able to handle increased traffic, more data, and additional functionality as the application grows.

For my application, one way I would plan for future growth would be to continue separating application responsibilities into individual services. Instead of having one large backend responsible for everything, different functions could be handled by separate microservices or Lambda functions.

For example, user management, trip information, payments, notifications, and reporting could eventually be separated into their own services. This would allow individual parts of the application to be changed or scaled without requiring the entire application to be changed at the same time.

Serverless architecture could also help manage growth. Lambda functions can execute when requests are received, which means resources do not have to remain running continuously just to wait for requests. API Gateway can provide a central entry point for those services, while S3 can continue to provide scalable storage for the frontend and other static resources.

Scale and Error Handling

As the application grows, I would need to plan for both increased traffic and failures.

For scaling, I would monitor application usage and identify which services receive the most traffic. Serverless services could automatically respond to increased demand, while containerized services could be scaled by increasing the number of running containers when necessary.

Error handling would also become more important. I would use logging and monitoring to identify failed requests and application errors. Lambda functions should return appropriate error responses instead of allowing failures to affect the entire application. API Gateway could also be used to provide consistent responses to clients.

For important operations, I would consider retries and appropriate timeout settings. I would also design services so that a failure in one part of the application does not automatically bring down the entire system.

Predicting Cloud Costs

Cost prediction would become increasingly important as the application grows.

I would estimate costs by looking at expected traffic, API requests, Lambda executions, storage requirements, database usage, and data transfer. Instead of simply estimating the total number of users, I would try to estimate how often each user interacts with the application.

For example, if an application has 10,000 users but each user only makes a few requests per month, the cost could be very different from an application with 1,000 users making hundreds of requests every day.

I would also monitor actual AWS usage after deployment and compare it with the original estimate. This would allow me to identify unexpected costs and adjust the architecture if necessary.

Containers vs. Serverless Cost Predictability

Containers and serverless have different cost advantages.

Containers can be more predictable because I can estimate the number of containers, their resource requirements, and how long they will run. If I know that I need a specific number of containers running continuously, I can estimate that infrastructure cost relatively easily.

Serverless can be less predictable because the cost depends more directly on usage. If traffic suddenly increases, the number of API requests and Lambda executions can also increase.

However, serverless can be more cost-efficient for applications with variable or unpredictable traffic because resources do not necessarily have to remain active when they are not being used.

For that reason, I would consider **containers more predictable for fixed workloads**, while **serverless can be more efficient for workloads that change significantly over time**.

The decision would depend on the application's traffic patterns and business requirements.

Advantages and Disadvantages of Expansion

There are several advantages to expanding the application using cloud-native services.

One advantage is scalability. Cloud services can provide additional resources as demand increases.

Another advantage is flexibility. Individual services can be updated or replaced without necessarily changing the entire application.

A third advantage is reduced infrastructure management. Services such as Lambda and S3 allow developers to focus more on application development instead of maintaining physical servers.

There are also disadvantages to this.

A larger microservice architecture can become more complicated to manage because there are more services, APIs, permissions, and points of failure.

Serverless applications can also create additional monitoring and debugging challenges because application logic may be spread across multiple functions.

Another concern is cost. Although serverless can be inexpensive at low usage levels, costs can increase as traffic increases. Without proper monitoring, unexpected usage can lead to unexpected charges.

There can also be a learning curve when working with AWS services. Developers need to understand IAM, networking, APIs, databases, monitoring, and security in addition to application development.

These factors would all be considered before expanding the application.

Elasticity and Pay-for-Service

Elasticity would play a major role in future growth.

If the application experiences a sudden increase in users, an elastic cloud architecture can respond to the increased demand instead of requiring me to manually purchase and configure new hardware.

This is especially useful for applications where traffic changes throughout the day or during specific events.

Pay-for-service would also influence my decisions. Instead of paying for a large amount of infrastructure before I know how much capacity is needed, I could use services where costs are more closely connected to actual usage.

However, I would still monitor costs carefully. Pay-for-use does not automatically mean low cost. If usage becomes extremely high, the total cost can also become high.

The best approach would be to balance scalability, performance, security, and cost rather than choosing a technology based on only one factor.

Conclusion

CS 470 helped me understand how software development extends beyond writing application code. A successful cloud application requires decisions about architecture, deployment, scalability, security, databases, and cost.

The biggest skills I developed were containerization, Docker Compose, AWS cloud services, serverless architecture, API integration, cloud storage, database concepts, and

cloud security. I also strengthened my problem-solving skills by troubleshooting development and deployment issues throughout the project.

Looking toward the future, I would continue improving the application by separating major functions into services, monitoring performance and costs, improving error handling, and using cloud elasticity to respond to changes in demand.

The biggest lesson I learned is that cloud development requires planning for the future. An application should not only work today; it should have an architecture that allows it to grow without becoming unnecessarily expensive or difficult to manage.

This course has given me a stronger foundation for pursuing software development and cloud-related positions. It also gives me a project that I can discuss with future employers because I can explain not only what I built, but why I made the architectural decisions I made and how I would improve the application as it grows.